

Activity 4

Fundamentals of Cryptography

6733172621 Patthadon Phengpinij

Collaborators. Claude (for L^AT_EX styling and grammar checking)

Overviews

In this activity, we will learn the basics of encryption. There are 3 exercises in this activity. Each exercise is designed to let you learn the concepts of cryptography. We will need:

- ImageMagick
- OpenSSL
- One of your favorite programming languages.

You are welcome to do this exercise with any programming language. If you have no preference, use Python.

Exercises

Problem 1. (Cryptanalysis)

Though encryption is primarily designed to preserve confidentiality and integrity of data, the mechanism itself is vulnerable to brute force (statistical analysis). In other words, the more we see the encrypted data, the easier we can hack it. In this exercise, you are asked to crack the following cipher text. Please provide the decrypted result and explain your strategy in decrypting this text.

Cipher text

PRCSOFQX FP QDR AFOPQ CZSPR LA JFPALOQSKR. QDFP FP ZK LIUBROJZK MOLTROE.

- a) Count the frequency of letters. List the top three most frequent characters.

Solution

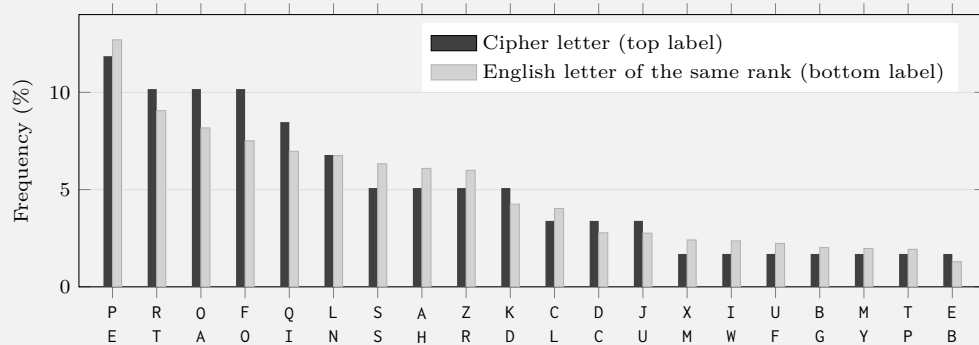
The cipher text has **59 letters** made of **20 distinct letters** (G, H, N, V, W, Y never appear). Counting every letter (notebook code/activity-03-code.ipynb, Task (a)) gives:

Cipher letter	P	R	O	F	Q	L	S	A	Z	K
Count	7	6	6	6	5	4	3	3	3	3
Frequency (%)	11.86	10.17	10.17	10.17	8.47	6.78	5.08	5.08	5.08	5.08
Cipher letter	C	D	J	X	I	U	B	M	T	E
Count	2	2	2	1	1	1	1	1	1	1
Frequency (%)	3.39	3.39	3.39	1.69	1.69	1.69	1.69	1.69	1.69	1.69

Top three most frequent characters:

1. P: 7 times (11.86%)
2. R, O, F: 6 times each (10.17%), a three-way tie
3. Q: 5 times (8.47%)

The chart below lines up the cipher letters, ranked by frequency, with the English letters of the same rank (ETAOIN SHRDLU...). Treating it as a direct lookup would guess P→E and R/O/F→T/A/O.



Letter frequency alone is not enough here. The message is very short: several letters share the same count, and the ranking can easily differ from typical English. (After cracking, P turns out to be S, not E.) So letter frequency is used only as a hint, e.g. “R and Q are frequent, so they may be E/T,” and the guesses are confirmed with the word analysis in (b).

- b) Knowing that this is English, what are commonly used three-letter words and two-letter words. Does the knowledge give you a hint on cracking the given text?

Solution

Common English words.

- **Three-letter:** THE, AND, FOR, ARE, BUT, NOT, YOU, ALL, ANY, CAN, HAD, HER, WAS, ONE, OUR, OUT, HAS, HIS, HOW, WHO. THE is by far the most common word in English.
- **Two-letter:** OF, TO, IN, IT, IS, BE, AS, AT, SO, WE, HE, BY, OR, ON, DO, IF, ME, MY, UP, AN, GO, NO, US, AM.

Word frequency of the cipher text. The word breaks and punctuation are kept in the cipher text, so the words can be counted as well. The *pattern* marks which positions repeat a letter. A monoalphabetic substitution never changes this pattern (e.g. PROVERB and MOLTROE both have the pattern ABCDEBF).

Cipher word	Length	Count	Pattern	Cipher word	Length	Count	Pattern
FP	2	2	AB	CZSPR	5	1	ABCDE
LA	2	1	AB	MOLTROE	7	1	ABCDEBF
ZK	2	1	AB	PRCSOFQX	8	1	ABCDEFGH
QDR	3	1	ABC	LIUBROJZK	9	1	ABCDEFGHI
QDFP	4	1	ABCD	JFPALQSKR	10	1	ABCDEFGHIJ
AFOPQ	5	1	ABCDE				

Yes, it gives strong hints. Without them we would have to search all $26! \approx 4.03 \times 10^{26}$ substitution keys. The word statistics cut this down to a handful of guesses:

- QDR is the *only* three-letter word, so it is most likely THE. This agrees with the letter counts from (a): R (6 times) and Q (5 times) are among the most frequent cipher letters, and E and T are the two most frequent English letters.
- QDFP starts with the same two letters as QDR, so it has the form TH__ (THIS, THAT, THEY, THEM, THUS, ...).
- FP is the only *repeated* word, and its last letter P is also the last letter of QDFP. A very common two-letter word that fits TH__ is IS, giving THIS IS.
- The other two-letter words LA and ZK can only be a few common words (OF, AN, ...), and MOLTROE has a repeated letter at positions 2 and 6. Both facts rule out most candidates once a few letters are known.

After T, H, E, I, S are fixed, every longer word is already partly readable. From there, the rest of the key follows word by word, as shown in (c).

- c) Cracking the given text. Measure the time that you have taken to crack this message.

Solution

Decrypted result.

SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN OLD GERMAN PROVERB.

Strategy. Combine the letter frequencies from (a) with the word analysis from (b). Guess one word at a time, starting with the words that have the fewest possible meanings. Every guess fixes some cipher letters, and those letters are substituted everywhere else, which makes the next word easier. To check each guess, the notebook's `candidates()` function lists all words in an English dictionary (ENABLE, 172,823 words) that have the same letter pattern *and* agree with the letters already solved.

Step	Cipher word	Guess	Reason	#Candidates
1	QDR	THE	The only 3-letter word; R, Q are frequent (E, T)	871
2	FP	IS	The repeated 2-letter word; also the ending of QDFP = TH__	59
3	QDFP	THIS	TH__ with four distinct letters, ending in S	1
4	PRCSOFQX	SECURITY	SE____IT_; SEDULITY and SENORITA also match, but only SECURITY fits the sentence	3
5	AFOPQ	FIRST	_IRST	1
6	CZSPR	CAUSE	C_USE	1
7	LA	OF	_F	1
8	JFPALOQSKR	MISFORTUNE	_ISFORTU_E	1
9	ZK	AN	A_, with N already known from step 8	1
10	MOLTROE	PROVERB	_RO_ER_ with the repeated R	1
11	LIUBROJZK	OLD GERMAN	O____ERMAN; not one dictionary word (two words written without a space), solved from context "…AN ____ PROVERB"	0

The partial plaintext after each step (_ = letter not solved yet):

```

1 _E____T_ __ THE ____T ____E ____T____E. TH__ __ ____E____E____
2 SE____IT_ IS THE _I_ST ____SE ____IS____T____E. THIS IS ____E____E____
3 SE____IT_ IS THE _I_ST ____SE ____IS____T____E. THIS IS ____E____E____
4 SECURITY IS THE _IRST C_USE ____IS____RTU____E. THIS IS ____ER____R____ER__
5 SECURITY IS THE FIRST C_USE _F _ISF_RTU____E. THIS IS ____ER____R____ER__
6 SECURITY IS THE FIRST CAUSE _F _ISF_RTU____E. THIS IS A ____ER_A ____R____ER__
7 SECURITY IS THE FIRST CAUSE OF _ISFORTU____E. THIS IS A _O____ER_A ____RO_ER__
8 SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN O____ERMAN _RO_ER__
9 SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN O____ERMAN _RO_ER__
10 SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN O____ERMAN PROVERB.
11 SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN OLDGERMAN PROVERB.

```

Recovered key. Writing the solved letters in plaintext-alphabet order shows the structure of the key. The cipher alphabet begins with ZECUR and then lists the remaining letters *in alphabetical order*. This means the text was encrypted with a **keyword substitution cipher** using the keyword ZECURE ("SECURE" with S replaced by Z). That structure also fills in the six letters that never appear in the message (shown in gray).

Plain	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Cipher	Z	E	C	U	R	A	B	D	F	G	H	I	J	K	L	M	N	O	P	Q	S	T	V	W	X	Y

Time taken.

- **By hand** (stopwatch, from first reading the cipher text to the full plaintext): **15 minutes**. The first three guesses are the hardest part (871 and 59 dictionary candidates). From step 3 on, every word has at most 3 candidates, so the rest is quick.
- **Dictionary-assisted** (replaying the 11 guesses with `candidates()` in the notebook): **0.908 seconds**.

This shows why a substitution cipher is weak. It hides the letters but keeps the letter frequencies, word lengths, and repetition patterns of the language. A longer cipher text would make these statistics even clearer and the text even easier to crack.

- d) Create a simple python program for cracking the Caesar cipher text using brute force attack. Explain the design and demonstrate your software. (You may use an English dictionary for validating results.)

Solution

Design. A Caesar cipher shifts every letter by the same secret amount k :

$$E_k(x) = (x + k) \bmod 26, \quad D_k(y) = (y - k) \bmod 26, \quad k \in \{0, 1, \dots, 25\}.$$

There are only 26 keys, so trying all of them is instant. The only hard part is recognizing which of the 26 outputs is the real message. The program (Task (d) in `code/activity-03-code.ipynb`) has four parts:

1. **Explicit key.** The secret shift is stored in the variable `KEY = 7`. `caesar_encrypt(text, key)` shifts each letter forward and leaves spaces and punctuation unchanged. `caesar_decrypt` is the same function with `-key`.
2. **Brute force.** `brute_force_caesar(ciphertext)` decrypts the cipher text with *every* key $0, \dots, 25$.
3. **Validation with an English dictionary.** `dictionary_score` is the fraction of letters that belong to words found in the ENABLE dictionary (weighting by letters stops short words that match by chance from inflating the score). Ties are broken with `chi_squared`, the χ^2 distance between the candidate's letter counts O_c and the counts E_c expected from English frequencies:

$$\chi^2 = \sum_{c \in \{A, \dots, Z\}} \frac{(O_c - E_c)^2}{E_c}.$$
4. **Decision.** The candidates are ranked by score. The best one is accepted only if its dictionary score is at least `threshold = 0.8`. Otherwise the program returns `None`, meaning the text is not a Caesar cipher.

Program.

```

1  import re
2  import time
3  import string
4  import statistics
5  from collections import Counter
6
7  import pandas as pd
8
9  ALPHABET = string.ascii_uppercase
10 KEY = 7 # secret shift of the Caesar cipher
11 MESSAGE = "SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN OLD GERMAN PROVERB."
12
13 # English dictionary (ENABLE word list) used to validate candidates
14 with open("enable1.txt", "r") as f:
15     ENGLISH_WORDS = {w.strip().upper() for w in f if w.strip()}
16
17 # Relative frequency (%) of each letter in typical English text
18 ENGLISH_FREQ = {
19     "E": 12.70, "T": 9.06, "A": 8.17, "O": 7.51, "I": 6.97, "N": 6.75, "S": 6.33,
20     "H": 6.09, "R": 5.99, "D": 4.25, "L": 4.03, "C": 2.78, "U": 2.76, "M": 2.41,
21     "W": 2.36, "F": 2.23, "G": 2.02, "Y": 1.97, "P": 1.93, "B": 1.29, "V": 0.98,
22     "K": 0.77, "J": 0.15, "X": 0.15, "Q": 0.10, "Z": 0.07,
23 }
24
25 def caesar_encrypt(text, key):
26     """Shift every letter 'key' positions forward in the alphabet; other characters are
27     kept."""
28     return "".join(ALPHABET[(ALPHABET.index(c) + key) % 26] if c in ALPHABET else c for c in
29                     text.upper())
30
31 def caesar_decrypt(text, key):
32     return caesar_encrypt(text, -key)

```

```

31
32 def dictionary_score(text):
33     """Fraction of letters that belong to valid English words (1.0 = every word is in the
    dictionary)."""
34     words = re.findall(r"[A-Z]+", text.upper())
35     total = sum(len(w) for w in words)
36     return sum(len(w) for w in words if w in ENGLISH_WORDS) / total if total else 0.0
37
38 def chi_squared(text):
39     """Chi-squared distance from English letter frequencies (lower = more English-like)."""
40     counts = Counter(c for c in text.upper() if c in ALPHABET)
41     n = sum(counts.values())
42     return sum((counts[c] - n * ENGLISH_FREQ[c] / 100) ** 2 / (n * ENGLISH_FREQ[c] / 100)
    for c in ALPHABET)
43
44 def brute_force_caesar(ciphertext, threshold=0.8):
45     """Try all 26 keys; return the ranking and (key, plaintext) of the best candidate, or
    None."""
46     rows = []
47     for key in range(26):
48         candidate = caesar_decrypt(ciphertext, key)
49         rows.append((key, dictionary_score(candidate), chi_squared(candidate), candidate))
50
51     ranking = pd.DataFrame(rows, columns=["key", "dictionary score", "chi-squared", "
    candidate plaintext"])
52     ranking = ranking.sort_values(["dictionary score", "chi-squared"], ascending=[False,
    True], ignore_index=True)
53
54     best = ranking.iloc[0]
55     answer = (int(best["key"]), best["candidate plaintext"]) if best["dictionary score"] >=
    threshold else None
56     return ranking, answer
57
58 # Demonstration 1: encrypt with KEY, then crack it without knowing KEY
59 caesar_cipher_text = caesar_encrypt(MESSAGE, KEY)
60 ranking, answer = brute_force_caesar(caesar_cipher_text)
61 recovered_key, recovered_text = answer
62 assert recovered_key == KEY and recovered_text == MESSAGE
63
64 # Demonstration 2: the same attack on the cipher text of this problem
65 ranking_given, answer_given = brute_force_caesar(
66     "PRCSOFQX FP QDR AFOPQ CZSPR LA JFPALQSKR. QDFP FP ZK LIUBROJZK MOLTROE."
67 )

```

Demonstration 1: cracking a Caesar cipher text. The message is encrypted with KEY = 7, and the program then recovers it from the cipher text alone:

```

Key          : 7
Message      : SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN OLD GERMAN PROVERB.
Cipher text  : ZLJBPAF PZ AOL MPYZA JHBZL VM TPZMVYABUL. AOPZ PZ HU VSK NLYTHU WYVCLYI.
Recovered key : 7
Recovered text : SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN OLD GERMAN PROVERB.

```

The first rows of the ranking (full 26-row table in the notebook):

Key	Dictionary score	χ^2	Candidate plaintext (first sentence)
7	1.000	15.347	SECURITY IS THE FIRST CAUSE OF MISFORTUNE.
14	0.102	573.317	LXVNBMR BL MAX YBKLM VTNLX HY FBLYHKMNGX.
1	0.068	1126.028	YKIXOZE OY ZNK LOXYZ IGAYK UL SOYLUXZATK.
20	0.034	228.747	FRPHEVGL VF GUR SVEFG PNHFR BS ZVFSBEGHAR.
⋮	⋮	⋮	⋮
25	0.000	1567.501	AMKCZQBG QA BPM NQZAB KICAM WN UQANWZBCVM.

The correct key is clearly separated from the others: score 1.000 versus at most 0.102 for every wrong key. It also has by far the lowest χ^2 . Over 100 repeated runs, the full 26-key attack takes **1.820 ms on average** (stdev 0.680 ms, min 1.318 ms, max 5.916 ms). As an extra check, the program recovers the correct key for all 26 possible values of KEY.

Demonstration 2: the given cipher text. Running the same attack on this problem's cipher text returns None:

Key	Dictionary score	χ^2	Candidate plaintext (first sentence)
23	0.102	53.397	SUFVRITA IS TGU DIRST FCVSU OD MISDORTVNU.
17	0.102	1149.848	YALBXOZG OY ZMA JOXYZ LIBYA UJ SOYJUXZBTA.
25	0.085	907.408	QSDTPGRY GQ RES BGPQR DATQS MB KGQBMPTLS.

No shift gives English (best score $0.102 < 0.8$), so the given text is *not* a Caesar cipher. This agrees with (c), where it turned out to be a keyword substitution cipher. The two key spaces explain why brute force works for one cipher but not the other. At the measured speed of about $1.820/26 \approx 0.07$ ms per key:

- **Caesar:** 26 keys, searched in about 1.8 ms.
- **General substitution:** $26! \approx 4.03 \times 10^{26}$ keys, which would take about 8.95×10^{14} years.

That is why the substitution cipher in (c) had to be broken with frequency and word analysis instead of trying every key.

Problem 2. (Cryptanalysis on Symmetric Encryption)

Vigenère is a complex version of the Caesar cipher. It is a polyalphabetic substitution.

- a) Please review [Kasiski examination](#). Explain how it can be used to attack Vigenère.

Solution

Setting. The Vigenère cipher shifts the i -th plaintext letter by the key letter at the same position, and the key of length m is repeated over the whole message:

$$C_i = (P_i + K_{i \bmod m}) \bmod 26, \quad P_i = (C_i - K_{i \bmod m}) \bmod 26.$$

Because the shift depends on the position, the same plaintext letter becomes different cipher letters (below, $A \rightarrow C/P$ and $T \rightarrow K/R$). This is why the letter-frequency attack from Problem 1 does not work directly.

Plain	A	T	T	A	C	K	A	T	D	A	W	N
Key	C	R	Y	P	T	O	C	R	Y	P	T	O
Cipher	C	K	R	P	V	Y	C	K	B	P	P	B

Demonstration setup. The notebook (Problem 2 in code/activity-03-code.ipynb) encrypts an 852-letter English paragraph with the key variable `VIGENERE_KEY = "CRYPTO"`. It then attacks the cipher text *without* using the key. The *index of coincidence*, $IC = \sum_c f_c(f_c - 1)/(N(N - 1))$, is the probability that two letters picked at random are equal. It shows how the encryption flattens the letter statistics:

Text	IC	Most frequent letters
English (expected)	0.0655	–
Random letters (expected)	0.0385	–
Plain text	0.0784	E 18.7%, T 11.2%, A 6.7%
Vigenère cipher text	0.0463	G 8.5%, C 7.7%, V 7.2%

The cipher text looks almost random (IC close to 0.0385), and its most frequent letter is far less dominant than E in the plain text.

1. Purpose of the Kasiski examination

The weakness of Vigenère is that the key *repeats*. The purpose of the Kasiski examination (published by Friedrich Kasiski in 1863) is to find the **key length** m from the cipher text alone. Knowing m is the turning point of the attack:

- All letters at positions $i \equiv j \pmod{m}$ are shifted by the same key letter. The polyalphabetic cipher therefore splits into m separate Caesar ciphers, and each one can be broken with the frequency analysis from Problem 1.

- Instead of searching 26^m keys ($26^6 \approx 3.09 \times 10^8$ for CRYPTO), the attacker only needs 26 trials per column, i.e. $26m = 156$ trials in total.

Why repeated fragments reveal m . Suppose the same plaintext fragment appears at positions i and j , and $j - i$ is a multiple of m . Then both copies are encrypted with the same sequence of key letters, so the *cipher text repeats as well*. If the copies start at different key positions, they are encrypted differently and do not repeat. So the distance between two identical cipher fragments is (very likely) a multiple of the key length.

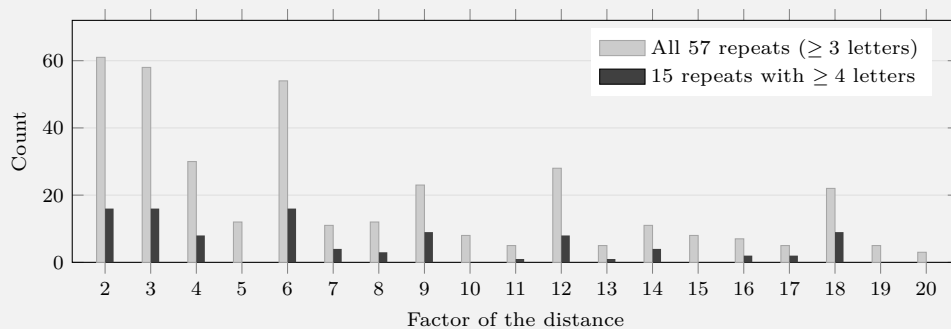
Procedure.

1. Find sequences of at least 3 letters that occur more than once in the cipher text, and record their positions.
2. Compute the distance between consecutive occurrences of each sequence.
3. Factor every distance and count how often each factor appears.
4. The key length is a factor shared by (almost) all distances. Usually it is the largest such factor, but the result is only a *candidate* and must be confirmed.

Worked example. The cipher text contains 57 repeated sequences, 15 of which have at least 4 letters. The plaintext column is shown only to explain the result; the attack never sees it.

#	Cipher sequence	Plaintext (hidden)	Positions	Distance(s)	Factorization
1	KFTVWRYCGMSZK	THECIPHERTEXT	463, 589	126	$2 \cdot 3^2 \cdot 7$
2	VYCHTAGG	THESAMEP	156, 444	288	$2^5 \cdot 3^2$
3	TGEAST	CIPHER	133, 211, 385	78, 174	$2 \cdot 3 \cdot 13, 2 \cdot 3 \cdot 29$
4	JTMHGI	LETTER	242, 284	42	$2 \cdot 3 \cdot 7$
5	MUMVG	OF THE	290, 542	252	$2^2 \cdot 3^2 \cdot 7$
6	QXSP	BEEN	21, 183	162	$2 \cdot 3^4$
7	TKPG	ERBE	147, 759	612	$2^2 \cdot 3^2 \cdot 17$
8	GUGC	EDIN	192, 786	594	$2 \cdot 3^3 \cdot 11$
9	NGGU	USED	298, 784	486	$2 \cdot 3^5$
10	VYYI	THAT	372, 738	366	$2 \cdot 3 \cdot 61$
11	ICEX	REPE	409, 817	408	$2^3 \cdot 3 \cdot 17$
12	IASO	THEM	423, 711	288	$2^5 \cdot 3^2$
13	PKFT	NTHE	504, 588	84	$2^2 \cdot 3 \cdot 7$
14	PIAS	RTHE	698, 710	12	$2^2 \cdot 3$
15	CKXF	EVER	776, 812	36	$2^2 \cdot 3^2$

Every one of these distances contains the factor $2 \cdot 3 = 6$, and their GCD is 6. Each repeat really is the same plaintext (e.g. THECIPHERTEXT, or CIPHER three times) at the same offset in the key. Counting the factors of the distances gives:



Limitations visible in the example.

- **Accidental repeats.** 19 of the 42 three-letter repeats are coincidences: different plaintexts that happen to encrypt to the same letters. For example, JRQ comes from HAS at position 12 and STB at position 31 (distance 19). With all repeats included, the GCD of the distances drops to 1, and factor 2 (61) and factor 3 (58) are counted more often than 6 (54). Longer repeats are much more reliable.

- **Divisors of the key length.** Every multiple of 6 is also a multiple of 2 and 3. So even among the clean 4-letter repeats, factors 2, 3 and 6 tie at 16. Kasiski narrows the key length to the candidates {2, 3, 6} (with 6 the most likely), and it needs enough cipher text to produce aligned repeats at all.

Both problems are resolved with **superimposition**.

2. Superimposition

Superimposition means writing the cipher text on top of (a shifted copy of) itself, so that letters encrypted with the *same key letter* line up. It serves two purposes in the attack: confirming the key length, and splitting the text into columns that can be solved one by one.

(i) **Superimposition test: confirming the key length.** Place the cipher text over a copy of itself shifted by s letters, and count the positions where the two letters are equal:

- If s is a **multiple of m** , both letters were shifted by the same key letter. They are equal exactly when their plaintext letters are equal, which happens with the English probability $\sum_c p_c^2 \approx 0.0655$.
- If s is **not a multiple of m** , the two shifts differ and a match is pure chance: $1/26 \approx 0.0385$.

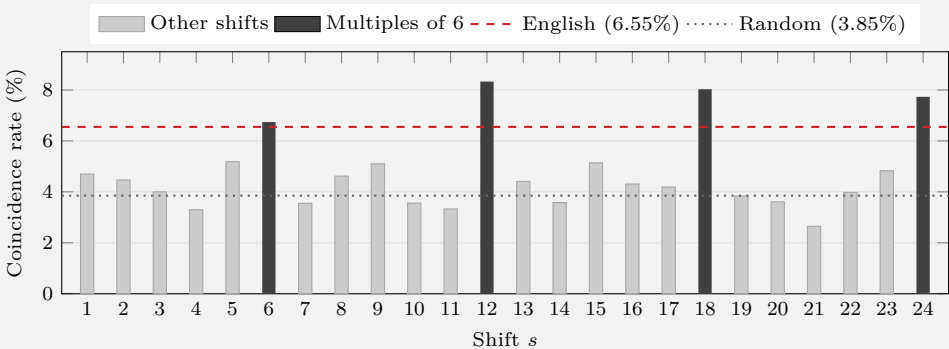
The first 36 letters, superimposed with a shift of 6, already show this. At every position where the cipher letters match (^), the hidden plaintext letters also match (R/R, P/P, S/S, T/T):

Cipher text : EIWEMCIIYEAMJRQPEKCPQQXSPRADGHGJRQXH

Shifted by s = 6 : IIYEAMJRQPEKCPQQXSPRADGHGJRQXHYVCCMV

Coincidence : ^ ^ ^ ^

Over the whole cipher text, the coincidence rate for each shift $s = 1, \dots, 24$ is:



Only the shifts 6, 12, 18 and 24 reach the English level (6.74–8.33%). All other shifts, including 2 (4.47%) and 3 (4.00%), stay near the random level. The GCD of the peaks is $\gcd(6, 12, 18, 24) = 6$, which rules out the Kasiski candidates 2 and 3.

The same idea can be applied to columns. Write the text in rows of a candidate length m , and average the IC of the m columns:

Candidate length m	1	2	3	4	5	6	7	8	9	10	11	12
Avg. column IC	0.046	0.051	0.060	0.051	0.046	0.078	0.046	0.051	0.060	0.051	0.044	0.080

$m = 6$ is the smallest length whose columns look like English. $m = 3$ only reaches 0.060, because each of its columns mixes two key letters (C/P, R/T, Y/O). $m = 12$ is also high, but only because it is a multiple of 6. The program therefore takes the smallest m whose average column IC is within 90% of the best, which gives $m = 6$.

(ii) **Superimposition by columns: recovering the key.** This is how Kasiski finished the attack. Write the cipher text in rows of $m = 6$ letters, which is the same as superimposing copies of the text shifted by 6, 12, 18, ... Every column now contains only letters that were shifted by *one* key letter k_j , so each of the 6 columns is an ordinary Caesar cipher with 142 letters:

Row	k_1	k_2	k_3	k_4	k_5	k_6
1	E	I	W	E	M	C
2	I	I	Y	E	A	M
3	J	R	Q	P	E	K
4	C	P	Q	Q	X	S
5	P	R	A	D	G	H
6	G	J	R	Q	X	H
⋮	⋮	⋮	⋮	⋮	⋮	⋮
142	Q	K	F	D	E	R

Each column is then solved like the Caesar cipher in Problem 1(d). A quick guess is that the column's most common letter stands for E. A more reliable test decrypts the column with all 26 shifts and keeps the one whose letter frequencies are closest to English (lowest χ^2):

Column	Letters	Most common	Key (most common = E)	Key (χ^2)	Best χ^2	2nd-best χ^2
1	142	G		C	48.99	369.10
2	142	V		R	46.09	380.07
3	142	C		Y	44.34	462.03
4	142	T		P	37.81	314.22
5	142	X		T	14.49	386.01
6	142	S		O	14.35	481.29

In every column, the best shift has a χ^2 that is 7–33 times smaller than the second best, so each key letter is unambiguous. The recovered key is **CRYPTO**. Decrypting with it gives back the original text exactly (CRYPTOGRAPHYHASALWAYSBEENACONTESTBETWEEN...). The complete attack (key length, then all 6 columns) takes **9.15 ms on average** (stdev 1.11 ms, 20 runs).

3. When does the attack work?

Running the same attack on only the first n letters of the cipher text:

Letters n	Letters per column	Key length found	Recovered key	Correct key letters
60	10	6	CRCLTO	4/6
120	20	6	CRYPTO	6/6
240	40	6	CRYPTO	6/6
480	80	6	CRYPTO	6/6
852	142	6	CRYPTO	6/6

- **Enough letters per column.** The period already shows up in 60 letters. But with only 10 letters per column, the frequency analysis picks the wrong shift for 2 columns. From about 20 letters per column onward, the key is recovered exactly. The attack needs roughly $20m$ letters, so a longer key needs proportionally more cipher text.
- **No period, no attack.** Both Kasiski and superimposition depend entirely on the key repeating. A key that is random, as long as the message, and never reused (a one-time pad) creates no aligned repeats and no coincidence peaks, so neither method finds anything.
- **Lesson.** Repetition leaks structure. A repeated key lets identical plaintext produce identical cipher text, which is the same weakness that ECB mode has in Problem 3.

Program (Problem 2 in the notebook; chi_squared and caesar_decrypt are reused from Problem 1(d)):

```

1  from collections import defaultdict
2
3  VIGENERE_KEY = "CRYPTO" # secret key of the Vigenere cipher
4
5
6  def only_letters(text):
7      return "".join(c for c in text.upper() if c in ALPHABET)
8
9
10 def vigenere_encrypt(text, key):
11     """C_i = (P_i + K_(i mod m)) mod 26 over the letters of `text`."""
12     letters = only_letters(text)
13     return "".join(ALPHABET[(ALPHABET.index(c) + ALPHABET.index(key[i % len(key)])) % 26]
14                     for i, c in enumerate(letters))

```

```

14
15
16 def vigenere_decrypt(text, key):
17     """P_i = (C_i - K_(i mod m)) mod 26."""
18     return "".join(ALPHABET[(ALPHABET.index(c) - ALPHABET.index(key[i % len(key)])) % 26]
19                     for i, c in enumerate(text))
20
21 # ---- Step 1: Kasiski examination -----
22 def repeated_sequences(ciphertext, min_len=3, max_len=20):
23     """Maximal repeated substrings (at least 'min_len' letters) and the positions where
24     they occur."""
25     seen = defaultdict(list)
26     for length in range(min_len, max_len + 1):
27         for i in range(len(ciphertext) - length + 1):
28             seen[ciphertext[i:i + length]].append(i)
29     repeats = {s: pos for s, pos in seen.items() if len(pos) > 1}
30
31     def extends(s, pos):
32         # s is only part of a longer repeat that occurs at exactly the same places
33         return any(len(repeats.get(s + c, ())) == len(pos) or len(repeats.get(c + s, ())) ==
34                    len(pos) for c in ALPHABET)
35
36     return {s: pos for s, pos in repeats.items() if not extends(s, pos)}
37
38 def factors(n, limit=20):
39     return [f for f in range(2, limit + 1) if n % f == 0]
40
41 # ---- Step 2: superimposition -----
42 def index_of_coincidence(text):
43     """Probability that two letters picked at random from 'text' are the same."""
44     counts = Counter(text)
45     n = len(text)
46     return sum(f * (f - 1) for f in counts.values()) / (n * (n - 1))
47
48 def coincidence_rate(ciphertext, shift):
49     """Superimpose the cipher text on itself shifted by 'shift'; percentage of aligned
50     letters that are equal."""
51     pairs = list(zip(ciphertext, ciphertext[shift:]))
52     return 100 * sum(a == b for a, b in pairs) / len(pairs)
53
54 def average_column_ic(ciphertext, length):
55     """Mean IC of the columns obtained by writing the cipher text in rows of 'length'
56     letters."""
57     return statistics.mean(index_of_coincidence(ciphertext[i::length]) for i in range(length))
58
59 def guess_key_length(ciphertext, max_len=12, ratio=0.9):
60     """Smallest length whose columns look (almost) as English as the best length; multiples
61     of the key score alike."""
62     scores = {m: average_column_ic(ciphertext, m) for m in range(1, max_len + 1)}
63     best = max(scores.values())
64     return min(m for m, score in scores.items() if score >= ratio * best)
65
66 # ---- Step 3: every column is a Caesar cipher -----
67 def solve_column(column):
68     """(chi-squared, shift) of all 26 Caesar shifts of one column, best first."""
69     return sorted((chi_squared(caesar_decrypt(column, s)), s) for s in range(26))
70
71
72

```

```

73 def kasiski_attack(ciphertext):
74     """Find the key length by superimposition, then solve every column as a Caesar cipher.
75     """
76     length = guess_key_length(ciphertext)
77     key = "".join(ALPHABET[solve_column(ciphertext[i::length])[0][1]] for i in range(length))
78     return key, vigenere_decrypt(ciphertext, key)
79
80 VIGENERE_CIPHER_TEXT = vigenere_encrypt(PLAIN_TEXT_2, VIGENERE_KEY)
81 recovered_key, recovered_plain = kasiski_attack(VIGENERE_CIPHER_TEXT)
82 assert recovered_key == VIGENERE_KEY and recovered_plain == only_letters(PLAIN_TEXT_2)

```

Problem 3. (Mode in Block Cipher)

Block Cipher is designed to have more randomness in a block. However, an individual block still utilizes the same key. Thus, it is recommended to use a cipher mode with an initial vector, chaining or feedback between blocks. This exercise will show you the weakness of Electronic Code Book mode which does not include any initial vector, chaining or feedback.

- a) Find a bitmap image that is larger than 2000x2000 pixels. Note that you may resize any image. To simplify the pattern, we will change it to bitmap (1-bit per pixel) using the portable bitmap format (pbm). In this example, we will use ImageMagick for the conversion.

```

1 $ magick convert image.jpg -resize 2000x2000 org.pbm
2 $

```

Solution

We drew our own test image with ImageMagick, step (a) of code/problem3.sh, instead of using a photo. It is 2048×2048 pixels, which is larger than 2000×2000 . This size is deliberate: a PBM row of 2048 pixels is $2048/8 = 256$ bytes, which is exactly 16 AES blocks of 16 bytes. So every AES block covers the same 128 pixels in every row. The picture contains two kinds of content:

- **Large flat areas:** the title CEDT 2110413, a padlock, and the word AES. Here many 16-byte blocks are identical.
- **A photo-like texture** on the left, made of dithered fractal noise. Here almost every block is different.

The conversion to a 1-bit PBM follows the handout. This machine has ImageMagick 6, where the command is convert (it is magick convert in version 7). The option -threshold 50% makes every pixel pure black or white:

```

$ convert image.png -resize 2048x2048 -threshold 50% org.pbm
$ identify org.pbm
org.pbm PBM 2048x2048 2048x2048+0+0 1-bit Bilevel Gray 524301B

```

CEDT 2110413



Original image org.pbm (2048×2048 pixels, 1 bit per pixel)

- b) The [NetPBM](#) format is a naive image format. The first two lines contain a header (format and size in pixel). Depending on the format, the pixels can be represented in either binary and ascii. For our exercise, we prefer binary. However, we first have to take out the header to prevent the encryption from encoding the header. To do so, use your text editor (eg. vi, notepad) to take out the first two lines.

```
1 $ cp org.pbm org.x
2 $ vi org.x
```

P4

2000 2000

KR)B0HD0000H0

Solution

A binary PBM (P4) file is a short text header followed by the raw pixels. Each pixel is 1 bit (1 = black, 0 = white), and every row is padded to a whole number of bytes:

Part	Content	Size
Line 1	P4 (magic number: binary bitmap)	3 bytes
Line 2	2048 2048 (width and height)	10 bytes
Pixels	2048 × 2048 bits, 256 bytes per row	524,288 bytes = 32,768 AES blocks

The header must be removed before encryption. Otherwise it would be encrypted too, and no image viewer could open the result. So only the pixel data is encrypted, and the plain header is put back afterwards in (d). We cut the header off with `head/tail` instead of `vi`, because a text editor can silently change binary data (for example, by adding a newline at the end of the file):

```
$ head -n 2 org.pbm > header.txt # "P4\n2048 2048\n" = 13 bytes
$ tail -c +14 org.pbm > org.x    # pixels only: 524301 - 13 = 524288 bytes
$ xxd -c 16 -l 48 org.x          # the top rows of the image are white
$
00000000: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000010: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000020: 0000 0000 0000 0000 0000 0000 0000 0000 .....
```

The top of the picture is white, so `org.x` starts with thousands of identical all-zero blocks. In total, only 3,280 of the 32,768 blocks are distinct, and the all-zero block alone appears 18,440 times (56%).

- c) Encrypt the file with [OpenSSL](#) with any block cipher algorithm in ECB mode (no padding and no salt).

```
1 $ openssl enc -aes-256-ecb -in org.x -nosalt -out enc.x
2 $
```

Solution

The pixel data is encrypted with AES-256 in ECB mode. The key is given explicitly as a variable, a 256-bit hex value passed with `-K`, instead of being derived from a password:

```
$ KEY=4d79af6618ab04423bd8e0e06f95f0c7156256d02e50380b1ed098d6adef1115
$ openssl enc -aes-256-ecb -e -in org.x -out enc-ecb.x -K $KEY -nopad -nosalt
$ xxd -c 16 -l 48 enc-ecb.x
00000000: 1fd8 a1dd 2b5f d9eb f4f5 76f5 47cb 3d96 ....+_....v.G.=.
00000010: 1fd8 a1dd 2b5f d9eb f4f5 76f5 47cb 3d96 ....+_....v.G.=.
00000020: 1fd8 a1dd 2b5f d9eb f4f5 76f5 47cb 3d96 ....+_....v.G.=.
```

- `-nopad`: the pixel data is already a whole number of blocks ($524,288 = 32,768 \times 16$), so no padding is needed. Without this option, OpenSSL would add a full extra block of PKCS#7 padding, and the file would no longer match the size in the header.
- `-nosalt`: with a password (no `-K`), `openssl enc` derives the key from the password and a random salt, and writes `Salted__` plus the 8-byte salt in front of the cipher text. Those

16 extra bytes would shift every pixel. With an explicit key there is no key derivation, so no salt is written.

The cipher text is exactly as large as org.x (524,288 bytes). Its first three blocks are *identical*: the three white plaintext blocks were all encrypted to the same cipher block 1fd8a1dd.... This is how ECB works: every block is encrypted on its own with the same key, $C_i = E_K(P_i)$.

d) Pad the header back and see the result.

```
1 $ cp org.pbm org.x
2 $ vi org.x
```

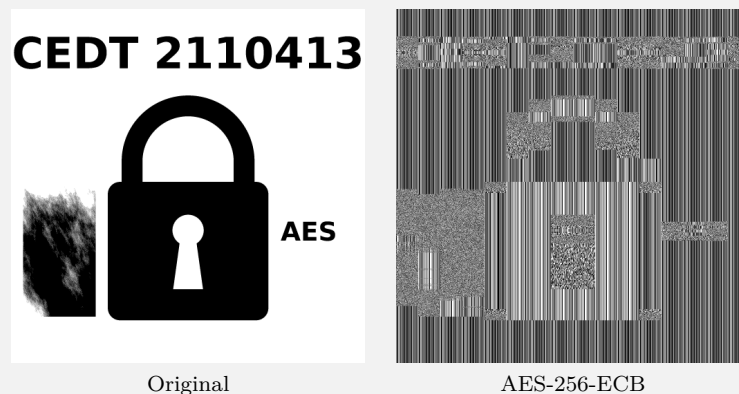
P4

```
2000 2000
KR)B@HD@
```

Solution

The plain header is put back in front of the cipher text (again with a shell command, not an editor), so the file is a valid PBM image again:

```
$ cat header.txt enc-ecb.x > ecb.pbm
$ identify ecb.pbm
ecb.pbm PBM 2048x2048 2048x2048+0+0 1-bit Bilevel Gray 524301B
```



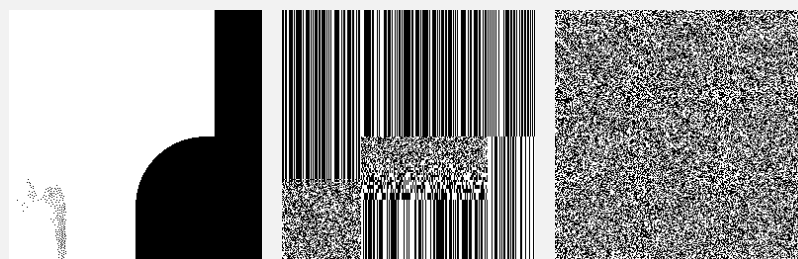
Original

AES-256-ECB

The picture is still recognizable after encryption. The title CEDT 2110413, the padlock with its keyhole, and the word AES are all clearly visible.

- Every **flat area** is made of identical plaintext blocks. These are encrypted to identical cipher blocks, which appear as one repeated stripe pattern. The white background becomes one pattern, and the black inside of the padlock becomes another.
- **Edges** and the **texture** on the left contain blocks that occur only once, so they look like random noise. Their *outline* is still visible, though, because the flat areas around them are not random.

A zoom into 256×256 pixels around the top-left corner of the padlock (shown at $2\times$):



Original

ECB

CBC (see (e))

In the PBM file, one AES block is a strip of $16 \times 8 = 128$ pixels within a single row. A row is exactly 16 blocks wide, so identical strips line up directly under each other and form vertical stripes, one pattern for each plaintext block value. With a width of 2000 pixels instead, a row would be 250 bytes, which is not a multiple of 16. The strips would then shift a little in every row, but the shapes would still be visible.

- e) You may try it with other modes with IV, chaining, or feedback and compare the result.

Solution

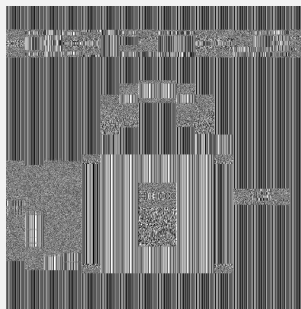
The same pixel data is encrypted with the same key in four modes that use an IV together with chaining or feedback. All of them use `-nopad -nosalt`, and the 128-bit IV is also given explicitly:

```
$ IV=ecd0a4e1eb4570bbd396dbccca0825
$ for mode in cbc cfb ofb ctr; do
  openssl enc -aes-256-$mode -e -in org.x -out enc-$mode.x -K $KEY -iv $IV -nopad -nosalt
  cat header.txt enc-$mode.x > $mode.pbm
done
```

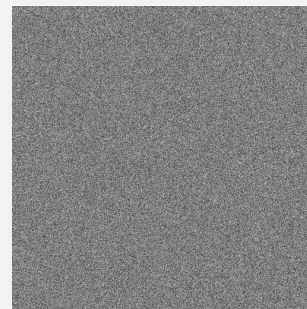
Every mode was also checked by decrypting with `-d`; each one gives back `org.x` byte for byte.



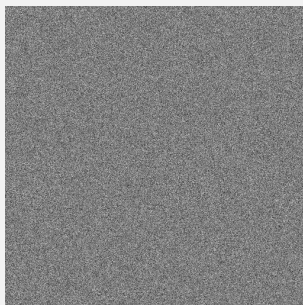
Original



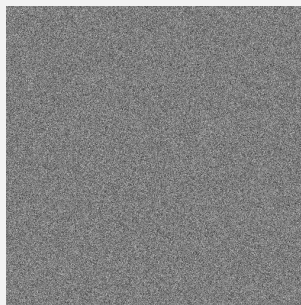
AES-256-ECB



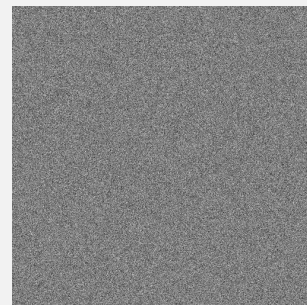
AES-256-CBC



AES-256-CFB



AES-256-OFB



AES-256-CTR

CBC, CFB, OFB and CTR all give uniform noise; nothing of the picture is left. Counting identical 16-byte blocks in each file puts an exact number on the difference:

File	Blocks	Distinct blocks	Most common block	Decrypts to org.x
org.x (plaintext)	32,768	3,280	18,440 times	—
ECB	32,768	3,280	18,440 times	yes
CBC	32,768	32,768	1 time	yes
CFB	32,768	32,768	1 time	yes
OFB	32,768	32,768	1 time	yes
CTR	32,768	32,768	1 time	yes

ECB keeps *exactly* the repetition structure of the plaintext: the same 3,280 distinct values, and the same count for the most common block. Every mode with an IV and chaining or feedback turns the 32,768 blocks into 32,768 different cipher blocks. The first CBC blocks show this directly. They come from the same all-zero plaintext blocks as in (c), yet they are all different:

```
$ xxd -c 16 -l 48 enc-cbc.x
$
```

```
00000000: ae1e 9f99 d677 f15d 6322 0639 6fe4 e35e  ....w.]c".9o..^
00000010: 7dbd d3ba 032d b526 00d8 cc58 fbb2 1903  }....-.&...X....
00000020: 19c6 fc61 b845 8fb5 2cc9 e3ee 0fec ff15  ...a.E.,.....
```

- f) What does the result suggest about the mode of operation in block cipher? Please provide your analysis.

Solution

Why ECB leaks. In ECB every block is encrypted on its own with the same key: $C_i = E_K(P_i)$. For a fixed key, E_K is a fixed permutation of the 2^{128} possible block values. Therefore $P_i = P_j \iff C_i = C_j$. The cipher text hides the *values* of the blocks, but it reveals exactly *which blocks are equal*. The experiment confirms this:

- The ECB file has the same 3,280 distinct blocks as the plaintext, and its most common block also occurs 18,440 times. An attacker learns the whole repetition structure without breaking AES.
- Any data with repeated blocks is exposed: flat image areas, long runs of zeros, file headers, fixed-format records, repeated database fields. In an image, the repetition structure *is* the picture.
- A larger key does not help. AES-256 was used, and the picture still shows, because the leak comes from the *mode*, not from the cipher.
- ECB is also deterministic and does not link blocks together. Encrypting the same message twice gives the same cipher text. An attacker can also reorder, delete or replay whole cipher blocks, and the result still decrypts without any error.

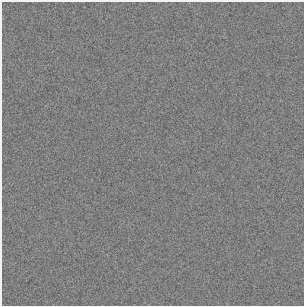
How an IV, chaining and feedback fix it. The other modes make each block's encryption depend on the IV and on either the previous blocks or the block's position. As a result, equal plaintext blocks no longer give equal cipher blocks (figure in (e)):

Mode	Encryption	What makes equal blocks differ	A bit error in C_i damages
ECB	$C_i = E_K(P_i)$	nothing (deterministic)	all of P_i
CBC	$C_i = E_K(P_i \oplus C_{i-1})$, $C_0 = \text{IV}$	random, unpredictable IV and chaining	all of P_i , one bit of P_{i+1}
CFB	$C_i = P_i \oplus E_K(C_{i-1})$, $C_0 = \text{IV}$	IV and feedback of the cipher text	one bit of P_i , all of P_{i+1}
OFB	$C_i = P_i \oplus O_i$, $O_i = E_K(O_{i-1})$, $O_0 = \text{IV}$	IV (key stream does not depend on the data)	the same bit of P_i only
CTR	$C_i = P_i \oplus E_K(\text{nonce} \parallel i)$	unique nonce and a counter	the same bit of P_i only

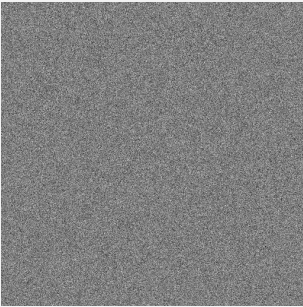
- **Padding:** ECB and CBC encrypt whole blocks, so they need padding when the data is not a multiple of 16 bytes. CFB, OFB and CTR use AES to build a stream cipher and need no padding.
- **Speed:** ECB and CTR can encrypt and decrypt all blocks in parallel, and CTR also allows random access. CBC and CFB encryption is sequential (their decryption can run in parallel), and OFB is sequential in both directions.

An IV only helps if it is used correctly. Two extra experiments in `code/problem3.sh` reuse the *same* key and IV:

- **CBC with a fixed IV is deterministic again.** Encrypting `org.x` twice gives byte-identical cipher texts, so an attacker can tell when the same message, or one with the same beginning, is sent twice. The IV must be random and unpredictable for every message.
- **CTR (and OFB) with a reused nonce produce the same key stream S twice.** Encrypting a second image (TOP SECRET) with the same key and IV gives $C_1 \oplus C_2 = (P_1 \oplus S) \oplus (P_2 \oplus S) = P_1 \oplus P_2$. The attacker gets the XOR of the two pictures without knowing the key:



$C_1 = \text{CTR}(\text{org.pbm})$



$C_2 = \text{CTR}(\text{second image})$



$C_1 \oplus C_2 = P_1 \oplus P_2$

Each cipher text on its own looks random, but their XOR shows both pictures on top of each other.

Conclusion.

- A strong block cipher is not enough on its own: the *mode of operation* decides whether patterns leak. ECB should not be used for data longer than one block.
- Use a mode with an IV and chaining or feedback, such as CBC with a random IV or CTR with a unique nonce. Never reuse an IV or nonce with the same key.
- These modes only protect confidentiality; none of them detects tampering. For example, in CTR an attacker can flip chosen plaintext bits by flipping cipher text bits. In practice, use an authenticated mode such as AES-GCM (CTR plus an authentication tag), which gives both confidentiality and integrity.

If you got it all right, the result should be like this.

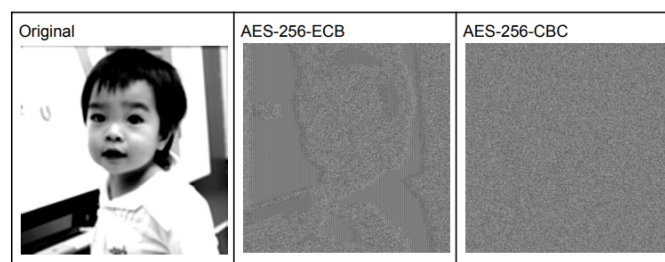


Figure 1: Example of ECB mode weakness

Problem 4. (Encryption Protocol - Digital Signature)

- a) Measure the performance of a hash function (sha1), RC4, Blowfish and DSA. Outline your experimental design. (Please use OpenSSL for your measurement)

Solution

Experimental Design

The performance of SHA-1, RC4, Blowfish, and DSA was measured using the OpenSSL speed command. All tests should be performed on the same machine and under similar system load to ensure a fair comparison.

Before the experiment, the OpenSSL version and CPU information can be recorded using:

```
1 $ openssl version
2 $ lscpu
```

Each test can be run for 10 seconds and repeated several times to obtain an average result.

SHA-1

SHA-1 performance is measured as hashing throughput:

```
1 $ openssl speed -elapsed -seconds 10 sha1
2 $
```

The result is reported in bytes/s or MB/s. Higher throughput means that more data can be hashed per second.

RC4

RC4 performance is measured as encryption throughput:

```
1 $ openssl speed -elapsed -seconds 10 rc4
2 $
```

For OpenSSL 3.x, the legacy provider may be required:

```
1 $ openssl speed -elapsed -seconds 10 \
2 $ -provider default -provider legacy rc4
```

Blowfish

Blowfish-CBC can be measured using:

```
1 openssl speed -elapsed -seconds 10 bf-cbc
```

For OpenSSL 3.x:

```
1 $ openssl speed -elapsed -seconds 10 \
2 $ -provider default -provider legacy bf-cbc
```

RC4 and Blowfish can be compared using their throughput in MB/s.

DSA

DSA is a digital signature algorithm, so its performance should not be measured in MB/s. Instead, the number of signing and verification operations per second is measured.

For example:

```
1 $ openssl speed -elapsed -seconds 10 dsa2048
2 $
```

The important results are:

- signatures per second,
- verifications per second.

Result Comparison

The results can be summarized as follows:

Algorithm	Metric	Result
SHA-1	Throughput at 16 KB	655.42 MB/s
RC4	Throughput at 16 KB	421.35 MB/s
Blowfish	Throughput at 16 KB	161.82 MB/s
DSA-2048	Signatures/s	2000.5
DSA-2048	Verifications/s	2500.8

SHA-1, RC4, and Blowfish are compared using data throughput, while DSA is evaluated using the number of signature operations per second because it performs a different type of cryptographic operation.

It should also be noted that SHA-1 and RC4 are considered obsolete for modern secure applications; this experiment measures their performance only.

- b) Comparing performance and security provided by each method.

Solution

Comparison of Performance and Security

The four algorithms provide different types of security services, so their performance should be compared together with their intended purpose.

Algorithm	Type	Performance	Security
SHA-1	Hash function	Very fast	Weak / obsolete
RC4	Stream cipher	Very fast	Insecure
Blowfish	Block cipher	Fast	Moderate, but outdated
DSA	Digital signature	Much slower	Strong if modern parameters are used

SHA-1

SHA-1 is generally fast because hashing requires only symmetric-style computations and does not involve expensive public-key operations.

However, SHA-1 is no longer considered secure for collision resistance. Practical collision attacks have been demonstrated, meaning that two different messages can be constructed with the same SHA-1 hash. Therefore, SHA-1 should not be used for modern digital signatures or security-sensitive applications.

RC4

RC4 usually has very high performance because it is a simple stream cipher and requires relatively few computations per byte.

However, RC4 contains several statistical weaknesses in its output stream and is considered insecure for modern cryptographic applications. Therefore, although RC4 may provide good performance, its security is insufficient.

Blowfish

Blowfish is a symmetric block cipher and generally provides good encryption throughput. It is usually slower than RC4 but still much faster than public-key algorithms such as DSA.

Blowfish itself has no practical attack against its full-round design when used correctly. However, it uses a relatively small 64-bit block size, which becomes problematic when encrypting large amounts of data. For this reason, newer ciphers such as AES are generally preferred.

DSA

DSA is much slower than SHA-1, RC4, and Blowfish because it performs computationally expensive public-key operations.

However, DSA provides a different security service. It provides:

- message authentication,

- message integrity, and
- digital signatures.

DSA does not provide confidentiality because it is not an encryption algorithm.

Its security depends on the key size, hash function, and correct generation of the per-signature random value. With appropriate parameters, DSA provides significantly stronger security properties than SHA-1 or RC4, although at a higher computational cost.

Conclusion

In general, there is a trade-off between performance and security functionality. SHA-1, RC4, and Blowfish are relatively fast because they use inexpensive operations, while DSA is slower because it uses public-key cryptography.

However, high performance does not necessarily mean good security. RC4 and SHA-1 are no longer considered secure, while Blowfish is outdated mainly because of its 64-bit block size. DSA can provide strong digital-signature security when appropriate parameters are used, but with significantly lower performance.

Therefore, the algorithms can be summarized as:

RC4/SHA-1/Blowfish: high throughput

DSA: lower performance but provides digital signatures

The choice of algorithm should therefore depend not only on performance, but also on the required security property.

- c) Explain the mechanism underlying Digital Signature. How does it combine the strength and weakness of each encryption scheme?

Solution

Mechanism of Digital Signature

A digital signature combines a **cryptographic hash function** with **asymmetric cryptography**. Its purpose is to provide message integrity, authentication, and non-repudiation.

The signing process can be summarized as follows:

1. The sender computes a hash of the message:

$$h = H(M),$$

where M is the message and H is a cryptographic hash function.

2. The sender uses their **private key** to generate a digital signature from the hash:

$$S = \text{Sign}_{K_{\text{private}}}(h).$$

3. The sender sends both the original message M and signature S to the receiver.

4. The receiver calculates the hash of the received message:

$$h' = H(M).$$

5. The receiver uses the sender's **public key** to verify the signature:

$$\text{Verify}_{K_{\text{public}}}(h', S).$$

If the verification succeeds, the receiver can conclude that the message has not been modified and that the signature was generated by the owner of the corresponding private key.

For DSA specifically, the hash is not simply “encrypted with the private key.” Instead, DSA uses mathematical operations involving the private key and a per-signature value to produce the signature. The basic idea, however, remains that the private key is required to create the signature while the public key can be used to verify it.

Combination of Strengths

Digital signatures combine the advantages of hashing and asymmetric cryptography.

- **Hash functions** such as SHA-1 are very fast and can reduce a message of any size into a small fixed-size digest. Therefore, the expensive public-key operation does not need to process the entire message.
- **Asymmetric algorithms** such as DSA allow the private key to remain secret while the public key can be distributed freely. This provides authentication and allows anyone with the public key to verify the signature.

Therefore, instead of applying a slow public-key operation to a large message, only the much smaller hash value is signed:

Large Message $\xrightarrow{\text{Fast Hash}}$ Small Digest $\xrightarrow{\text{Public-Key Signature}}$ Digital Signature.

This gives both good efficiency and strong authentication.

Weaknesses

The security of a digital signature depends on both components. If either component is weak, the entire signature system may become insecure.

For example, SHA-1 has known collision attacks. If an attacker can create two different messages with the same hash,

$$H(M_1) = H(M_2),$$

a signature associated with one message may potentially be abused with another specially constructed message. Therefore, modern systems use stronger hash functions such as SHA-256 instead of SHA-1.

DSA also depends on secure private-key management and correct generation of its per-signature value. If the private key or this value is exposed or generated incorrectly, the signature security can be broken.

Relation to Symmetric Encryption

Symmetric algorithms such as RC4 or Blowfish are very fast and are suitable for encrypting large amounts of data, but both communicating parties must share the same secret key. They mainly provide **confidentiality**, not digital signatures.

A practical secure communication system can therefore combine both approaches:

Digital Signature $\rightarrow$ Authentication + Integrity

Symmetric Encryption $\rightarrow$ Confidentiality.

For example, a message may first be digitally signed and then encrypted using a symmetric cipher. This combines the high performance of symmetric encryption with the authentication and integrity provided by digital signatures.

Conclusion

Digital signatures combine the speed of cryptographic hash functions with the authentication capability of asymmetric cryptography. Hashing makes the process efficient, while public-key cryptography allows signatures to be verified without sharing the private key.

However, the overall security is limited by the weakest component. A weak hash function, insecure private key, or incorrectly implemented signature algorithm can compromise the complete digital-signature system. Digital signatures also do not provide confidentiality by themselves, so encryption must be used separately when confidentiality is required.

Hint: (OpenSSL command line)

```
1 # List algorithms
2 $ openssl list cipher-algorithms
3 # To encrypt
4 $ openssl enc -ciphername [options] -e -in filename -out filename \
5   -K key -iv IV -nopad -nosalt
```